

System Blueprint: Multi-View React Tab Routing Setup

Architectural Concept Model for Lightweight, State-Driven Workspace Navigation

Architectural Concept Model

To smoothly map out additional workspace views inside a single dashboard structure without introducing the overhead or complexity of an external router library like `react-router-dom`, the application handles route context natively using a single active view identifier state (`currentView`).

The navigation layout sidebar renders conditional CSS styles depending on whether a given navigation item matches the active view identifier, while the core main container section mounts only the matching target component layout branch dynamically.

Step 1: Centralized View Index Config

To keep the application modular as you add rows of new tabs (like Audit logs, System Settings, Case analytics, etc.), you can define your navigation layout structures into a dynamic map array configuration directly above or inside your dashboard component context:

```

const NAVIGATION_ITEMS = [
  {
    id: "evidence",
    label: "Evidence Repository",
    icon: (
      <svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24"
strokeWidth="2" ...>
        <path strokeLinecap="round" strokeLinejoin="round" d="M20.25 14.15v4.25c0 .
594..." />
      </svg>
    )
  },
  {
    id: "identity",
    label: "Identity Management",
    icon: (
      <svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24"
strokeWidth="2" ...>
        <path strokeLinecap="round" strokeLinejoin="round" d="M19 7.5v3m0 0v3m0-3h3..." />
      </svg>
    )
  },
  {
    id: "audit",
    label: "System Audit Logs",
    icon: (
      <svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24"
strokeWidth="2" ...>
        <path strokeLinecap="round" strokeLinejoin="round" d="M9 12h6m-6 4h6m2 5H7..." />
      </svg>
    )
  }
];

```

Step 2: Dynamic Mapping of the Navigation Component

Instead of writing long lists of structural layout loops for desktop navigation views and duplicating them inside mobile drawer blocks, using a list configuration map ensures you can cleanly update your sidebar options using one single element row loop:

```

<nav className="space-y-2 mt-16">
  {NAVIGATION_ITEMS.map((item) => {
    const isSelected = currentView === item.id;
    return (
      <button
        key={item.id}
        onClick={() => {
          setCurrentView(item.id);
          setIsMobileMenuOpen(false); // Clean drawer dismissal on phone views
        }}
        className={`w-full text-left px-4 py-3 rounded-xl font-bold text-sm transition-all
flex items-center gap-3 border ${
          isSelected
            ? "bg-blue-600/10 text-blue-400 border-blue-500/20 shadow-lg shadow-
blue-950/10"
            : "bg-transparent text-slate-400 border-transparent hover:bg-slate-800/50
hover:text-slate-200"
          }`}
      >
        {item.icon}
        {item.label}
      </button>
    );
  })}
</nav>

```

Step 3: Conditional Component Target Workspace Switch

Inside your standard `<main>` layout container view component layer area, handle layout mounting safely using conditional logic gates:

```

{/* MAIN CONTAINER PANEL */}
<main className="flex-1 p-4 sm:p-8 overflow-y-auto max-w-5xl w-full mx-auto space-y-8">

  {/* VIEW INDEX 1: EVIDENCE SUB-SYSTEM ROUTE */}
  {currentView === "evidence" && (
    <div className="animate-fadeIn space-y-6">
      <h1 className="text-2xl font-black text-white">Evidence Management</h1>
      {/* Evidence Dashboard Code or Subcomponent Goes Here */}
    </div>
  )}

  {/* VIEW INDEX 2: IDENTITY WORKSPACE ROUTE */}
  {currentView === "identity" && (
    <div className="animate-fadeIn space-y-6">
      <h1 className="text-2xl font-black text-white">Identity Access Management</h1>
      {/* Account Creation Form & Directory List Table Elements */}
    </div>
  )}

  {/* VIEW INDEX 3: SYSTEM LOGGING WORKSPACE ROUTE */}
  {currentView === "audit" && (
    <div className="animate-fadeIn space-y-6">
      <h1 className="text-2xl font-black text-white">System Audit Ledger</h1>
      {/* Database Query Action Log Listing UI Elements */}
    </div>
  )}

</main>

```

Process to Append Future Tabs

Whenever your interface requirements expand to support a brand new workspace feature branch, follow these two steps:

1. **Insert a configuration row:** Define a new metadata object layout containing a chosen unique string id key inside the NAVIGATION_ITEMS array configuration block.
2. **Formulate a layout target:** Map a matching condition block ({currentView === "your_new_key" && (. . .)}) inside the central rendering workspace layout area of your UI code.